VISUAL BASIC .NET

Unit-1: Introduction to VB.NET

Comprehensive Study Notes

Table of Contents

1. Introduction to Visual Basic

Visual Basic (VB) is an **object-oriented programming language** developed by Microsoft as part of the .NET framework. It is a **successor** to the original Visual Basic 6.0 and is designed to be **easy to learn** while being **powerful** enough for professional application development.

Types of Applications You Can Build

Application Type	Description	Examples
Windows Forms	Desktop GUI applications built with drag-and-drop controls, event-driven model	Calculator, Notepad, CRM Software, Inventory System
Console Applications	Command-line interface applications, no GUI	Batch processing, Automation scripts, System utilities
ASP.NET Web Apps	Web-based applications using ASP.NET Web Forms or MVC	E-commerce, Social media, Banking portals
Class Libraries (DLLs)	Reusable code components for sharing functionality	Math library, Data access layer, Validation library
Mobile Applications	Cross-platform mobile apps using Xamarin / .NET MAUI	Shopping apps, Fitness trackers, Messaging apps
Windows Services	Background services without UI, run on system boot	Auto-backup service, Email notification service
WPF Applications	Modern desktop apps with XAML, advanced graphics and animations	Media players, Design tools, Data visualization apps

2. Features of VB.NET

#	Feature	Description
1	**Rapid Application Development (RAD)**	Drag-and-drop interface for visual form design. Controls can be dragged from the toolbox onto the form, drastically reducing UI development time.
2	**Event-Driven Programming**	Program flow is determined by user actions (clicks, keypresses, mouse movements). Code executes only when events occur.
3	**Object-Oriented Programming (OOP)**	Supports Encapsulation, Inheritance, Polymorphism, and Abstraction.
4	**Garbage Collection**	CLR automatically manages memory. Objects no longer in use are freed automatically, preventing memory leaks.
5	**Type Safety**	Strongly typed language. Variables must be declared with a specific type. Compiler checks types at compile time.
6	**Language Interoperability**	VB code can call code written in C#, F#, or any .NET language. All compile to MSIL.
7	**Common Language Runtime (CLR)**	Provides common execution environment with memory management, security, and exception handling.
8	**Extensive Class Library (FCL)**	Thousands of pre-built classes for file I/O, networking, database access, UI, graphics, and more.
9	**Structured Exception Handling**	Try-Catch-Finally blocks for graceful handling of runtime errors.
10	**My Namespace**	Unique VB namespace providing easy access to My.Computer, My.Application, My.Settings, etc.
11	**LINQ**	Language Integrated Query for querying data from collections, databases, XML.
12	**Multi-threading**	Support for parallel execution using System.Threading and Async/Await.
13	**Windows Forms**	Framework for building rich desktop applications with graphical user interface.

14	**Web Development**	ASP.NET integration for building web applications, services, and APIs.
15	**Database Access**	ADO.NET, Entity Framework, LINQ to SQL for data access.
16	**Security**	Code Access Security (CAS), role-based security, cryptography support.
17	**Deployment**	ClickOnce, MSI Installer, XCopy deployment options.
18	**Debugging**	Breakpoints, Watch windows, Locals, Immediate Window, Edit and Continue.
19	**Community & Ecosystem**	Large developer community, extensive documentation, NuGet package manager.

VB.NET vs Other .NET Languages

Feature	VB.NET	C#	C++/CLI	F#
Syntax Style	English-like, uses keywords (Dim, Sub, End)	C-style, uses symbols ({ }, ;)	C++ style with extensions	Functional programming style
Case Sensitivity	Case-insensitive	Case-sensitive	Case-sensitive	Case-sensitive
Event Handling	Uses Handles keyword	Uses += operator	Uses += operator	Uses += operator
Type Inference	Dim x = 10 (Option Infer On)	var x = 10;	auto	Automatic
My Namespace	Available	Not available	Not available	Not available
Learning Curve	Easy for beginners	Moderate	Difficult	Moderate
Late Binding	Allowed (Option Strict Off)	Not allowed (use dynamic)	Not allowed	Not applicable

3. .NET Framework Architecture

Compilation and Execution Flow

```
SOURCE CODE (.vb files)
       |
       v
COMPILATION PHASE (vbc.exe)
  - Lexical Analysis
  - Syntax Parsing
  - Semantic Analysis
  - Code Generation
       |
       v
MSIL GENERATION (Microsoft Intermediate Language)
  - Platform-independent bytecode
  - Contains instructions, metadata, manifest
       |
       v
ASSEMBLY CREATION (EXE or DLL)
  - Bundles MSIL with metadata and manifest
  - Contains version, security permissions, references
       |
       v
JIT COMPILATION (At Runtime - per method)
  - Converts MSIL to Native Machine Code
  - Specific to current processor (x86, x64)
  - Cached in memory for future use
       |
       v
CLR EXECUTION
  - Memory Management (Garbage Collection)
  - Security (Code Access Security)
  - Exception Handling
  - Thread Management
  - Type Safety
       |
       v
OUTPUT - Application Runs!
```

Key .NET Framework Concepts

Concept	Explanation

MSIL (Microsoft Intermediate Language)	CPU-independent set of instructions, similar to Java bytecode. Makes code portable across any system with .NET installed.
JIT Compiler	Converts MSIL to native machine code just before execution. Optimizations based on current environment. Only called methods get compiled.
CLR (Common Language Runtime)	Execution engine for .NET applications. Provides memory management, security, exception handling, and thread management.
Garbage Collection	Automatic memory management. Identifies objects no longer in use and frees their memory. Prevents memory leaks.
CTS (Common Type System)	Standardized type system ensuring all .NET languages use the same data types. Defines rules for type declarations, usage, and management.
CLS (Common Language Specification)	Set of rules for .NET language interoperability. Ensures code written in one language can be used by another language.
BCL (Base Class Library)	Foundation classes for .NET: System, System.IO, System.Data, System.Net, System.Windows.Forms, etc.

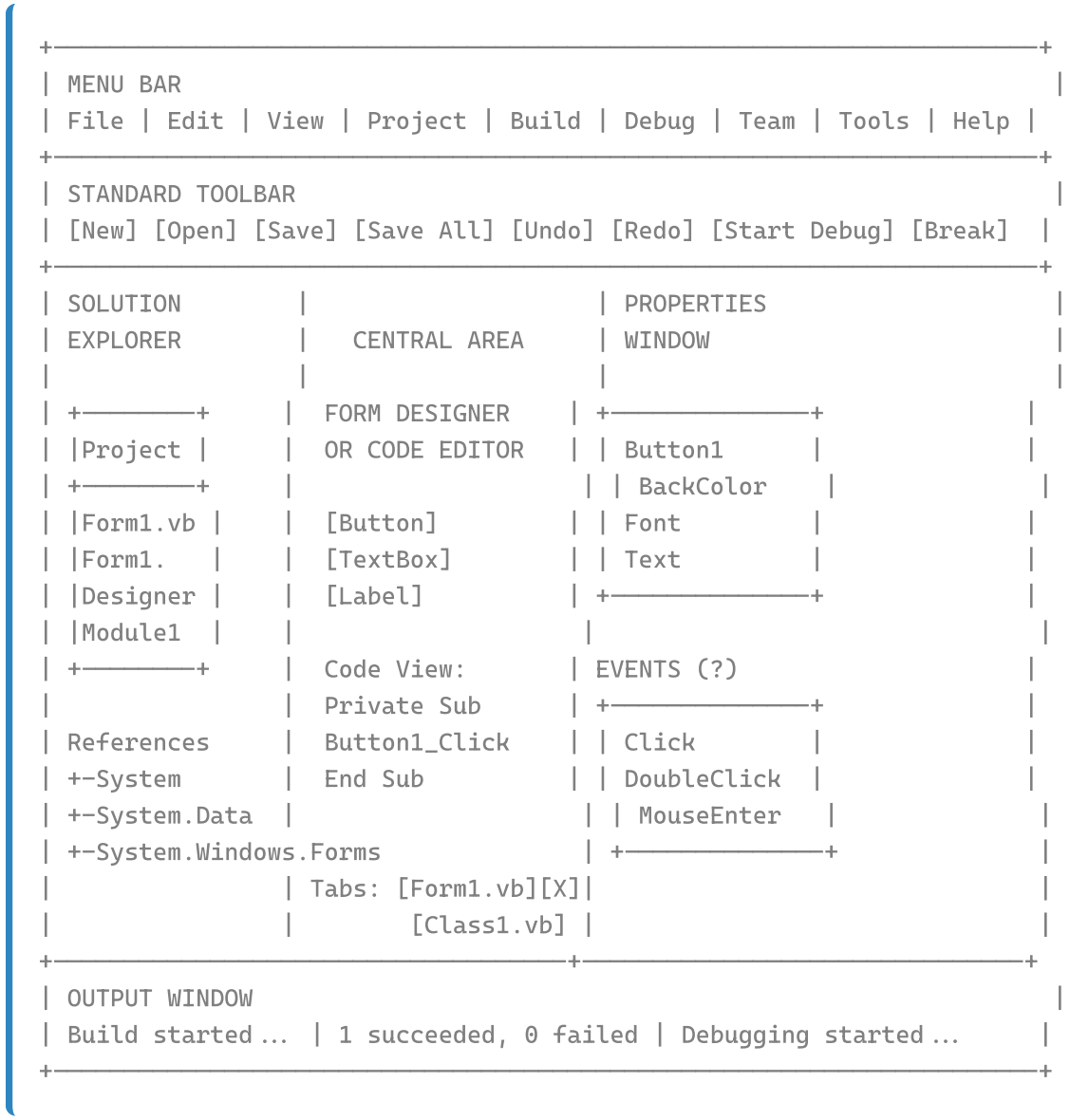
4. Visual Studio IDE

Visual Studio is an **Integrated Development Environment (IDE)** created by Microsoft. It provides comprehensive tools for developing, debugging, and deploying applications for Windows, web, cloud, and mobile platforms.

IDE Components

Component	Description
Menu Bar	File, Edit, View, Project, Build, Debug, Team, Tools, Window, Help. Contains all IDE functions.
Toolbox	Contains controls (Button, Label, TextBox, ComboBox, ListBox, etc.) that can be dragged onto the form. Organized into categories: Common Controls, Containers, Data, Components, Dialogs.
Solution Explorer	Displays solution/project hierarchy. Shows files, references, and allows adding/removing items. Right-click context menu for operations.
Properties Window	Shows properties of selected object (control/form). Properties can be modified at design-time. Events tab shows available events.
Form Designer	Visual design surface for creating UI. WYSIWYG interface. Drag-and-drop controls from Toolbox.
Code Editor	For writing code with IntelliSense, syntax highlighting, code snippets. Multiple tabs for open files. Split view for designer and code side-by-side.
Output Window	Displays build output, debug messages, errors. Can switch between different output sources.
Error List	Shows errors, warnings, and messages. Double-click to navigate to error location. Filter by error type.
Immediate Window	Execute VB code on-the-fly during debugging. Evaluate expressions: ? variableName. Change variable values.
Watch Window	Monitor variables and expressions. See values change as code executes. Multiple watch windows possible.
Locals Window	Shows all local variables in current scope. Updates during execution. Expand objects to see properties.

IDE Layout Diagram



5. Windows Forms

Form Properties

Property	Description	Example Values
Name	Unique identifier for the form in code	frmMain, frmLogin
Text	Text displayed in the form's title bar	"My Application", "Login"
BackColor	Background color of the form	Color.White, Color.Blue
BackgroundImage	Image displayed as form background	Image.FromFile("bg.jpg")
Size	Width and height of the form in pixels	800, 600
Location	Position of the form on screen (X, Y)	100, 50
StartPosition	Where the form appears when shown	CenterScreen, CenterParent, Manual
WindowState	How the form is displayed initially	Normal, Maximized, Minimized
FormBorderStyle	Style of the form's border and title bar	FixedSingle, Sizable, None, Fixed3D, FixedDialog
ControlBox	Show/hide minimize, maximize, close buttons	True, False
MaximizeBox	Enable/disable maximize button	True, False
MinimizeBox	Enable/disable minimize button	True, False
Opacity	Transparency level (0% to 100%)	0.5 (50% transparent)

TopMost	Form stays on top of all other windows	True, False
AcceptButton	Button clicked when Enter key is pressed	btnSubmit, btnOK
CancelButton	Button clicked when Escape key is pressed	btnCancel, btnClose
AutoScaleMode	How the form scales with DPI changes	None, Font, Dpi, Inherit
Font	Default font for all controls on the form	New Font("Arial", 12)
ForeColor	Default text color for controls	Color.Black, Color.White

Control Properties

Property	Description	Example Values
Name	Unique identifier for the control	btnSubmit, txtName, lblStatus
Text	Display text shown on the control	"Click Me", "Enter Name:"
BackColor	Background color of the control	Color.Red, SystemColors.Control
ForeColor	Color of text on the control	Color.White, Color.Black
Font	Font family, size, and style for text	New Font("Arial", 12, FontStyle.Bold)
Size	Width and height in pixels	100, 30
Location	Position on the form (X, Y)	50, 50
Anchor	Which edges the control anchors to	Top, Left; Bottom, Right
Dock	How the control attaches to form edges	Top, Fill, Bottom, Left, Right, None
Enabled	Can the user interact with the control?	True, False
Visible	Is the control shown on the form?	True, False
TabIndex	Tab order position	0, 1, 2

TabStop	Can you tab to this control?	True, False
Cursor	Cursor style when mouse is over control	Cursors.Hand, Cursors.WaitCursor, Cursors.IBeam

6. Common Controls

Control	Purpose	Key Properties	Key Events
Button	Trigger actions when clicked	Text, Enabled, BackColor, Font, Image	Click, DoubleClick, MouseEnter, MouseLeave
Label	Display static text (non-editable)	Text, Font, ForeColor, AutoSize, TextAlign	Click, DoubleClick
TextBox	User text input (editable)	Text, Multiline, MaxLength, ReadOnly, PasswordChar	TextChanged, KeyPress, Enter, Leave
ComboBox	Dropdown selection list	Items, SelectedIndex, SelectedItem, DropDownStyle	SelectedIndexChanged, DropDown
ListBox	List of selectable items	Items, SelectedIndex, SelectionMode, MultiColumn	SelectedIndexChanged, Click
CheckBox	On/Off toggle switch	Checked, Text, CheckState, Appearance	CheckedChanged, CheckStateChanged
RadioButton	Mutually exclusive option	Checked, Text	CheckedChanged
PictureBox	Display images	Image, SizeMode, ImageLocation, BorderStyle	Click, LoadCompleted
Timer	Execute code at intervals	Enabled, Interval (milliseconds)	Tick
DateTimePicker	Select date	Value, Format, MinDate, MaxDate	ValueChanged

	and/or time		
MonthCalendar	Display and select dates from a calendar	SelectionStart, SelectionEnd, MaxDate, MinDate	DateChanged, DateSelected
ProgressBar	Show task progress	Value, Maximum, Minimum, Style	-
TrackBar	Select numeric value from a range	Value, Minimum, Maximum, TickFrequency	ValueChanged, Scroll
NumericUpDown	Select numeric values with spin buttons	Value, Minimum, Maximum, Increment, DecimalPlaces	ValueChanged
RichTextBox	Rich text editing with formatting	Text, Rtf, SelectionFont, SelectionColor	TextChanged, LinkClicked
WebBrowser	Display web pages within the form	Url, DocumentText, IsWebBrowserContextMenuEnabled	Navigated, DocumentCompleted

Setting Properties in Code vs Designer

```
' Setting properties in CODE (at runtime)
Private Sub Form1_Load(sender As Object, e As EventArgs) Handles MyBase.Loac
    Me.Text = "My Application"
    Me.BackColor = Color.White
    Me.Size = New Size(800, 600)
    btnSubmit.Text = "Click Me"
    btnSubmit.BackColor = Color.Blue
    btnSubmit.ForeColor = Color.White
```

```
    btnSubmit.Font = New Font("Arial", 12, FontStyle.Bold)
End Sub

' Setting properties in DESIGNER (Form1.Designer.vb)
' These are automatically generated by the IDE:
Private Sub InitializeComponent()
    Me.btnSubmit = New Button()
    Me.btnSubmit.Text = "Submit"
    Me.btnSubmit.Location = New Point(50, 50)
    Me.btnSubmit.Size = New Size(100, 30)
    Me.Controls.Add(Me.btnSubmit)
End Sub
```

7. Event-Driven Programming

In VB.NET, the program flow is determined by **user actions** (events) like clicking a button, typing in a textbox, or moving the mouse. Code executes **only when events occur**, making the application responsive to user interactions.

OOP Concepts: Properties, Methods, Events

Concept	Description	Example
Properties	Attributes or characteristics of an object. Describe the STATE of an object. Have Get and Set accessors.	Button1.Text = "Click" Dim color = Button1.BackColor
Methods	Actions or behaviors that an object can perform. Procedures (Sub or Function) associated with an object.	Button1.PerformClick() TextBox1.Clear()
Events	Actions that occur in response to user interaction. Event handlers are Sub procedures with the Handles keyword.	Button1.Click TextBox1.TextChanged

Common Form Events

Event	When It Occurs	Typical Use
Load	Before the form is displayed for the first time	Initialize form, load data from database, set default values
Shown	After the form is displayed for the first time	Show welcome messages, start animations
FormClosing	When the form is about to close (can be cancelled)	Confirm exit, save unsaved data
FormClosed	After the form has closed (cannot be cancelled)	Cleanup resources, log activity
Activated	When the form receives focus	Refresh data, update UI, start timers
Deactivate	When the form loses focus	Save state, pause operations

Resize	When the form is resized	Adjust control layout, reposition elements
Paint	When the form needs to be redrawn	Custom drawing, graphics operations
KeyPress	When a character key is pressed	Character validation, input filtering
KeyDown	When any key is pressed	Keyboard shortcuts, input handling
KeyUp	When a key is released	Keyboard shortcut release handling
MouseMove	When mouse moves over the form	Drag operations, hover effects
MouseDown	When mouse button is pressed	Start drag operations, selection
MouseUp	When mouse button is released	End drag operations, execute actions
MouseEnter	When mouse enters the form area	Highlight form, start hover effects
MouseLeave	When mouse leaves the form area	Remove highlights, stop effects
Click	When form is clicked	Simple actions, selection

Common Control Events

Control	Event	When It Occurs
Button	Click	Triggered when button is clicked by mouse or Enter key
TextBox	TextChanged	Triggered when text content changes
TextBox	KeyPress	Triggered when a character key is pressed
TextBox	Enter / Leave	Triggered when control receives/loses focus
ComboBox	SelectedIndexChanged	Triggered when selection changes
ListBox	SelectedIndexChanged	Triggered when selection changes
CheckBox	CheckedChanged	Triggered when checked state changes
RadioButton	CheckedChanged	Triggered when checked state changes
DateTimePicker	ValueChanged	Triggered when date/time value changes

Timer	Tick	Triggered at specified interval
PictureBox	Click	Triggered when picture box is clicked
DataGridView	CellClick	Triggered when a cell is clicked

Event Handler Syntax

```
' Button Click Event
Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.
    MessageBox.Show("Button was clicked!")
End Sub

' Form Load Event
Private Sub Form1_Load(sender As Object, e As EventArgs) Handles MyBase.Loac
    Me.Text = "Welcome"
    TextBox1.Text = "Enter your name"
End Sub

' TextBox TextChanged Event
Private Sub TextBox1_TextChanged(sender As Object, e As EventArgs) Handles T
    Label1.Text = "You typed: " & TextBox1.Text
End Sub

' ComboBox SelectedIndexChanged Event
Private Sub ComboBox1_SelectedIndexChanged(sender As Object, e As EventArgs)
    MessageBox.Show("You selected: " & ComboBox1.SelectedItem.ToString())
End Sub

' MouseEnter and MouseLeave Events
Private Sub Button1_MouseEnter(sender As Object, e As EventArgs) Handles But
    Button1.BackColor = Color.LightBlue
End Sub

Private Sub Button1_MouseLeave(sender As Object, e As EventArgs) Handles But
    Button1.BackColor = SystemColors.Control
End Sub

' KeyPress Event
Private Sub TextBox1_KeyPress(sender As Object, e As KeyPressEventArgs) Hanc
    If Not Char.IsDigit(e.KeyChar) AndAlso Not Char.IsControl(e.KeyChar) The
        e.Handled = True  ' Only allow digits
    End If
End Sub

' Form Resize Event
Private Sub Form1_Resize(sender As Object, e As EventArgs) Handles MyBase.Re
    Label1.Text = "Size: " & Me.Width.ToString() & " x " & Me.Height.ToStrir
```

```
End Sub

' FormClosing Event (with confirmation)
Private Sub Form1_FormClosing(sender As Object, e As FormClosingEventArgs) H
    Dim result As DialogResult = MessageBox.Show("Are you sure you want to e
        "Confirm Exit", MessageBoxButtons.YesNo, MessageBoxIcon.Question)
    If result = DialogResult.No Then
        e.Cancel = True
    End If
End Sub

' Paint Event (custom drawing)
Private Sub Form1_Paint(sender As Object, e As PaintEventArgs) Handles MyBas
    Dim g As Graphics = e.Graphics
    g.DrawString("Hello VB.NET", New Font("Arial", 24),
        Brushes.Blue, 100, 100)
End Sub
```

8. First Program & Code Structure

Hello World Example

```
' 1. Imports Section
Imports System
Imports System.Windows.Forms
Imports System.Drawing

' 2. Namespace Declaration
Namespace MyApplication

    ' 3. Class Declaration
    Public Class Form1
        Inherits System.Windows.Forms.Form

        ' 4. Field Declarations
        Private counter As Integer = 0
        Private userName As String

        ' 5. Constructor
        Public Sub New()
            InitializeComponent()
        End Sub

        ' 6. Designer-generated code
        Private Sub InitializeComponent()
            Me.Label1 = New Label()
            Me.TextBox1 = New TextBox()
            Me.Button1 = New Button()

            Me.Label1.Text = "Enter your name:"
            Me.Label1.Location = New Point(20, 20)

            Me.TextBox1.Location = New Point(20, 50)
            Me.TextBox1.Size = New Size(200, 20)

            Me.Button1.Text = "Click Me"
            Me.Button1.Location = New Point(20, 80)

            Me.Controls.Add(Me.Label1)
            Me.Controls.Add(Me.TextBox1)
            Me.Controls.Add(Me.Button1)

            Me.Text = "Hello World Application"
            Me.Size = New Size(300, 200)
        End Sub
```

```
        ' 7. Event Handlers
        Private Sub Button1_Click(sender As Object, e As EventArgs) Handles
            counter += 1
            Dim name As String = TextBox1.Text
            If String.IsNullOrEmpty(name) Then
                name = "World"
            End If
            MessageBox.Show("Hello " & name & "! You clicked " &
                counter.ToString() & " time(s).")
        End Sub

        ' 8. Sub Procedures
        Private Sub ClearForm()
            TextBox1.Text = ""
            counter = 0
        End Sub

        ' 9. Function Procedures
        Private Function ValidateInput() As Boolean
            Return Not String.IsNullOrEmpty(TextBox1.Text)
        End Function

    End Class

End Namespace
```

VB Form Code Structure

1. **Imports Section** - Import namespaces (System, System.Windows.Forms, System.Drawing)
2. **Namespace Declaration** - Organize related classes together
3. **Class Declaration** - Define the Form class inheriting from System.Windows.Forms.Form
4. **Field Declarations** - Class-level variables
5. **Event Handlers** - Sub procedures with Handles keyword responding to events
6. **Sub Procedures** - Helper methods performing actions without returning values
7. **Function Procedures** - Methods that return values
8. **Properties** - Expose data with Get/Set accessors

Code Readability Standards

Practice	Good Example	Bad Example
Meaningful Names	btnSubmit, txtCustomerName	Button1, TextBox1
Consistent Indentation	4 spaces per level	No indentation
Capitalization	PascalCase methods, camelCase variables	Random casing
Comments	Explain WHY code exists	Obvious "Add 1 to counter"

Commenting Guide

```
' 1. SINGLE-LINE COMMENT
' This is a single-line comment using apostrophe

' 2. INLINE COMMENT
Dim total As Integer = 10 * 5  ' This calculates the total

' 3. XML DOCUMENTATION COMMENT
''' <summary>
''' Calculates the total price including tax.
''' </summary>
''' <param name="price">The base price</param>
''' <returns>Total price including tax</returns>
Private Function CalculateTotal(ByVal price As Double) As Double
    Return price * 1.1
End Function

' 4. TODO COMMENT (Shows in Task List)
'TODO: Implement error handling for this method

' 5. REGION COMMENT (Code collapse)
#Region "Event Handlers"
#End Region
```

Project File Structure

```
MyProject/                  Root folder
 +-- MyProject.sln          Solution file
 +-- MyProject.vbproj        Project file
 +-- My Project/             Project properties
 +-- Form1.vb                Form code file
 +-- Form1.Designer.vb       Designer-generated code
 +-- Form1.resx              Form resource file
```

```
+-- App.config              Application configuration
+-- bin/Debug/              Debug build output
+-- bin/Release/            Release build output
+-- obj/                    Intermediate files
+-- packages/               NuGet packages
```

9. The My Namespace

"My" is a unique namespace in Visual Basic that provides easy access to commonly used .NET Framework features with intuitive objects for common tasks.

Category	Purpose	Example
My.Application	Access application-level information	My.Application.Info.AssemblyName My.Application.Info.Version
My.Computer	Access computer resources (hardware, OS)	My.Computer.FileSystem.WriteAllText() My.Computer.Clipboard.SetText() My.Computer.Audio.Play() My.Computer.Network.IsAvailable
My.Resources	Access embedded application resources	My.Resources.Logo My.Resources.WelcomeMessage
My.Settings	Access application/user settings	My.Settings.UserName My.Settings.Save()
My.User	Access current user information	My.User.Name My.User.IsAuthenticated
My.Forms	Easy access to forms in the project	My.Forms.Form2.Show()
My.WebServices	Access web services	My.WebServices.Service1.GetData()

My.Computer Examples

```
' File System Operations
My.Computer.FileSystem.WriteAllText("file.txt", data, False)
Dim content As String = My.Computer.FileSystem.ReadAllText("file.txt")
My.Computer.FileSystem.DeleteFile("file.txt")
Dim exists As Boolean = My.Computer.FileSystem.FileExists("test.txt")

' Clipboard
My.Computer.Clipboard.SetText("Hello")
Dim text As String = My.Computer.Clipboard.GetText()

' Audio
My.Computer.Audio.Play("sound.wav")
My.Computer.Audio.Play("sound.wav", AudioPlayMode.Background)
```

```vb
' System Information
Dim computerName As String = My.Computer.Name
Dim osName As String = My.Computer.Info.OSFullName
Dim totalMemory As Long = My.Computer.Info.TotalPhysicalMemory

' Registry
My.Computer.Registry.SetValue("HKCU\Software\MyApp", "KeyName", "Value")
Dim value As Object = My.Computer.Registry.GetValue("HKCU\Software\MyApp", '
```

10. Debugging

Types of Errors

Type	Description	Example
Syntax Error	Violations of language grammar rules. Caught at compile time.	Missing End If, misspelled keywords, mismatched quotes
Runtime Error	Errors during program execution. Application may crash if not handled.	Division by zero, File not found, Null reference
Logical Error	Code runs but produces incorrect results. Hardest to detect.	Wrong formula, incorrect condition, wrong loop bounds

Visual Studio Debugging Windows

Window	Purpose	Shortcut
Locals Window	Shows all local variables in current scope	Debug ? Windows ? Locals
Watch Window	Monitor specific variables or expressions	Debug ? Windows ? Watch
Autos Window	Variables used in current and previous lines	Debug ? Windows ? Autos
Call Stack Window	Shows method call hierarchy	Debug ? Windows ? Call Stack
Output Window	Build and debug output messages	View ? Output (Ctrl+Alt+O)
Immediate Window	Execute code during debugging	Debug ? Windows ? Immediate (Ctrl+Alt+I)
Breakpoints Window	Manage all breakpoints	Debug ? Windows ? Breakpoints

Debugging Process

1. **Set Breakpoints** - Click in margin, press F9, or choose Debug ? Toggle Breakpoint

2. **Start Debugging** - Press F5 (Start Debugging) or Ctrl+F5 (Start Without Debugging)
3. **Examine State** - Hover over variables, Watch Window, Locals Window
4. **Step Through Code** - F11 (Step Into), F10 (Step Over), Shift+F11 (Step Out)
5. **Repair and Continue** - Edit code while debugging, continue with changes

Debugging Example

```
Imports System.Diagnostics  ' For Debug.WriteLine

Private Sub DebuggingExample()
    Dim total As Integer = 0

    ' Set breakpoint on the line below
    For i As Integer = 1 To 10
        total += i
        ' Use Output window to trace values
        Debug.WriteLine("i = " & i.ToString() & ", total = " & total.ToStrir
    Next

    MessageBox.Show("Total: " & total.ToString())
End Sub
```

Edit and Continue Feature

Edit and Continue allows you to edit code while paused in debug mode and apply changes without restarting the debugging session. Supported changes include modifying method bodies, adding/removing local variables, and changing property values. Not supported: adding methods, changing signatures, or adding classes.

11. Quick Revision Points

#	Point
1	Visual Basic is an object-oriented programming language for the .NET framework
2	RAD - Rapid Application Development with drag-and-drop UI design
3	Event-Driven - Code responds to user actions (clicks, keypresses)
4	MSIL - Microsoft Intermediate Language (platform-independent bytecode)
5	CLR - Common Language Runtime (execution environment)
6	JIT - Just-In-Time compiler (converts MSIL to native code)
7	Garbage Collection - Automatic memory management
8	Solution (.sln) - Contains one or more projects
9	Project (.vbproj) - Contains files and settings for a single application
10	Form Designer - Visual design surface for UI creation
11	Controls - UI elements (Button, TextBox, Label, etc.)
12	Properties - Attributes of objects (Text, Size, Color)
13	Methods - Actions that objects can perform
14	Events - Actions that occur in response to user interactions
15	Event Handler - Code that responds to an event
16	Handles keyword - Associates a method with an event
17	Syntax Error - Violation of language rules (caught at compile time)
18	Runtime Error - Error during program execution
19	Logical Error - Program runs but produces incorrect results
20	Breakpoint - Pauses execution at a specific line
21	Step Into (F11) - Executes code line by line
22	Step Over (F10) - Executes method without stepping into it

23	Watch Window - Monitor variables and expressions
24	Locals Window - Shows all local variables
25	Call Stack - Shows method call hierarchy
26	Immediate Window - Execute code during debugging
27	Edit and Continue - Edit code while debugging
28	My Namespace - Easy access to common features
29	Code Snippets - Reusable code templates (if, for, foreach, try, etc.)
30	Smart Compile - Auto-corrects common errors

Questions

1. **What is Visual Basic .NET? Explain its key features in detail.**
   - Discuss at least 10 major features with examples
   - How is VB.NET different from classic Visual Basic 6.0?
2. **Explain the .NET Framework architecture with a neat diagram.**
   - What are MSIL, CLR, JIT, CTS, and CLS?
   - Describe the compilation and execution process of a VB.NET program
3. **Compare VB.NET with C# and F#. What are the advantages of choosing VB.NET?**
   - Discuss syntax, case sensitivity, event handling, type inference
   - Explain the unique features of VB.NET (My Namespace, Handles keyword)
4. **Describe the various components of Visual Studio IDE.**
   - Menu Bar, Toolbox, Solution Explorer, Properties Window
   - Form Designer, Code Editor, Output Window, Error List
5. **What are Windows Forms? Explain common form properties and control properties.**
   - List at least 15 form properties with their descriptions
   - Explain the difference between setting properties at design-time vs runtime
6. **Explain the common controls available in VB.NET with their purposes and key events.**
   - Button, Label, TextBox, ComboBox, ListBox, CheckBox, RadioButton
   - PictureBox, Timer, DateTimePicker, ProgressBar, RichTextBox
7. **What is Event-Driven Programming? Explain with examples.**
   - Define event, event handler, and the Handles keyword
   - List and explain 10 common form events with their usage
8. **Write a complete "Hello World" VB.NET program with proper code structure.**
   - Include Imports, Namespace, Class, Event Handlers
   - Explain each section of the code
9. **Explain the My Namespace in VB.NET with examples.**
   - My.Application, My.Computer, My.Resources, My.Settings, My.User
10. **Discuss debugging techniques in Visual Studio.**
    - Types of errors: Syntax, Runtime, Logical

- Breakpoints, Step Into/Over, Watch Window, Locals Window
- Immediate Window, Call Stack, Edit and Continue